RASHMI M
1DS24RI404

Multi-sensor dashboard (simulated IMU + temp + proximity)

1. Aim :

To design and implement a real-time multi-sensor monitoring dashboard using a web-based interface that visualizes simulated sensor data including accelerometer, gyroscope, temperature, and proximity readings.

2. Theory :

In embedded and IoT systems, continuous monitoring of sensor data is essential for diagnostics, control, and analysis. A dashboard provides a user-friendly interface to visualize real-time data streams.

This system simulates multiple sensors:

- **Accelerometer (Acc X, Y, Z):** Measures acceleration forces along three axes.
- **Gyroscope (Gyro X, Y, Z):** Measures angular velocity.
- **Temperature Sensor:** Monitors environmental temperature.
- **Proximity Sensor:** Detects distance or presence of nearby objects.

To simulate real-world variations:

- **Sine and cosine functions** are used to create periodic patterns.
- **Random noise** is added to mimic sensor inaccuracies.

A sliding window approach using a **deque (double-ended queue)** ensures that only recent data points (fixed size) are stored and visualized.

Assumptions

- Sensor data is simulated and not from actual hardware.
- Noise added is uniformly random.
- Sampling interval is fixed at 500 ms.
- No communication delays or hardware limitations are considered.

3. System Architecture :

The system is a **web-based interactive dashboard** built using Dash and Plotly.

Inputs:

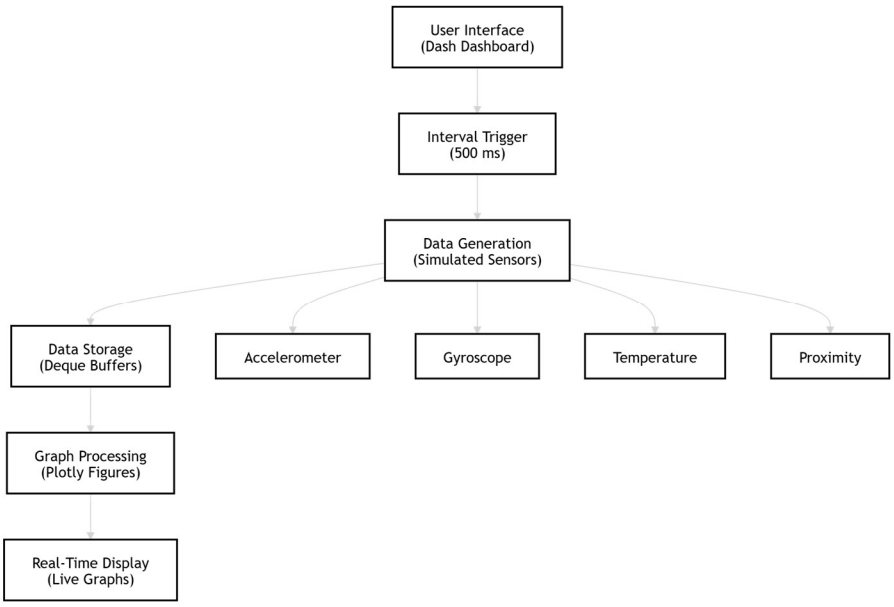
- Simulated time intervals
- Random noise values
- Mathematical functions (sine/cosine)

Processing Module:

This module performs:

- **Data Generation:**
  - Accelerometer: sin() and cos() functions + noise
  - Gyroscope: Random values between -1 and 1
  - Temperature: Base value + sinusoidal fluctuation
  - Proximity: Sinusoidal variation + noise
- **Data Storage:**
  - Uses deque with fixed maximum length (50)
- **Graph Updating:**
  - Triggered every 500 ms using Dash Interval
  - Updates all sensor plots dynamically

4. Block Diagram :



5. Code :

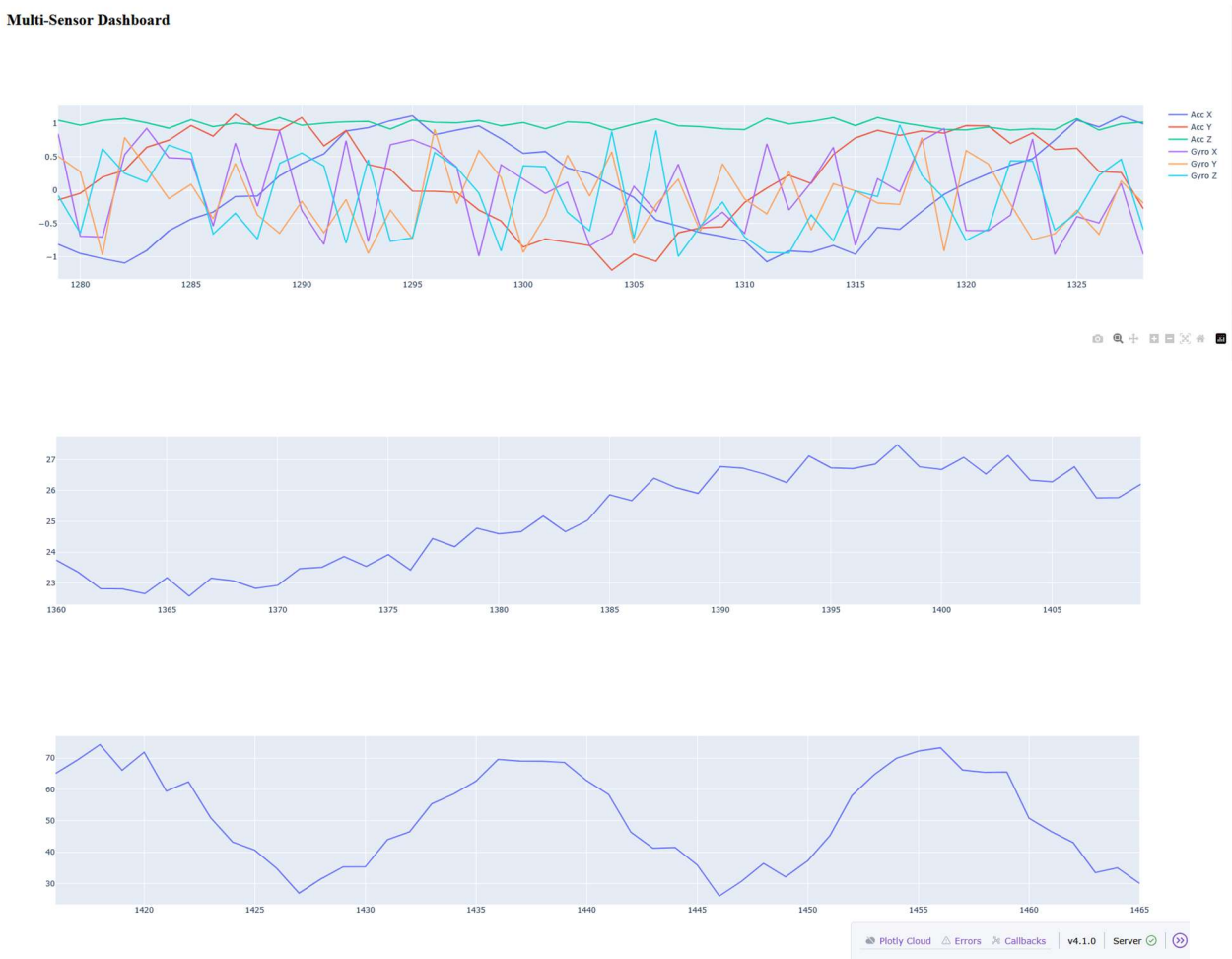
```
import random
import math
from collections import deque
from dash import Dash, dcc, html
from dash.dependencies import Input, Output
import plotly.graph_objs as go

app = Dash(__name__)

max_len = 50

acc_x = deque(maxlen=max_len)
acc_y = deque(maxlen=max_len)
acc_z = deque(maxlen=max_len)
gyro_x = deque(maxlen=max_len)
gyro_y = deque(maxlen=max_len)
gyro_z = deque(maxlen=max_len)
temp = deque(maxlen=max_len)
prox = deque(maxlen=max_len)
t = deque(maxlen=max_len)

for i in range(max_len):
    acc_x.append(0)
    acc_y.append(0)
    acc_z.append(0)
    gyro_x.append(0)
    gyro_y.append(0)
    gyro_z.append(0)
    temp.append(25)
    prox.append(50)
    t.append(i)

app.layout = html.Div([
    html.H2("Multi-Sensor Dashboard"),
    dcc.Graph(id='imu-graph'),
    dcc.Graph(id='temp-graph'),
    dcc.Graph(id='prox-graph'),
    dcc.Interval(id='interval', interval=500, n_intervals=0)
])

@app.callback(
    [Output('imu-graph', 'figure'),
```

```
     Output('temp-graph', 'figure'),
     Output('prox-graph', 'figure')],
    [Input('interval', 'n_intervals')]
)
def update_graph(n):
    time = n

    acc_x.append(math.sin(time/5) + random.uniform(-0.2,0.2))
    acc_y.append(math.cos(time/5) + random.uniform(-0.2,0.2))
    acc_z.append(1 + random.uniform(-0.1,0.1))

    gyro_x.append(random.uniform(-1,1))
    gyro_y.append(random.uniform(-1,1))
    gyro_z.append(random.uniform(-1,1))

    temp.append(25 + math.sin(time/10)*2 + random.uniform(-0.5,0.5))
    prox.append(50 + math.sin(time/3)*20 + random.uniform(-5,5))

    t.append(time)

    imu_fig = go.Figure([
        go.Scatter(x=list(t), y=list(acc_x), mode='lines', name='Acc X'),
        go.Scatter(x=list(t), y=list(acc_y), mode='lines', name='Acc Y'),
        go.Scatter(x=list(t), y=list(acc_z), mode='lines', name='Acc Z'),
        go.Scatter(x=list(t), y=list(gyro_x), mode='lines', name='Gyro X'),
        go.Scatter(x=list(t), y=list(gyro_y), mode='lines', name='Gyro Y'),
        go.Scatter(x=list(t), y=list(gyro_z), mode='lines', name='Gyro Z')
    ])

    temp_fig = go.Figure([
        go.Scatter(x=list(t), y=list(temp), mode='lines', name='Temperature (°C)')
    ])

    prox_fig = go.Figure([
        go.Scatter(x=list(t), y=list(prox), mode='lines', name='Proximity')
    ])

    return imu_fig, temp_fig, prox_fig

if __name__ == '__main__':
    app.run(debug=True)
```

6. Input Values :

- Sampling Interval = 500 ms
- Max Data Points = 50
- Initial Temperature = 25 °C
- Initial Proximity = 50 units
- Noise Range:
    - Accelerometer: ±0.2
    - Gyroscope: ±1
    - Temperature: ±0.5
    - Proximity: ±5

7. Output :



- Real-time plots of:
    - Accelerometer (X, Y, Z)
    - Gyroscope (X, Y, Z)
    - Temperature (°C)

    - Proximity
- Smooth and continuously updating graphs
- Visualization of sensor trends and fluctuations

8. Conclusion :

The implemented dashboard provides an effective way to simulate and visualize multi-sensor data in real time. It demonstrates how web-based tools can be used for monitoring embedded systems without physical hardware.

While suitable for simulation and learning purposes, real-world implementations would require:

- Integration with actual sensors
- Data filtering techniques
- Handling communication delays and noise more realistically